

Dataflow Labeling Convention

Purpose

This convention standardizes how dataflows are labeled in Data Flow Diagrams (DFDs).

Goals:

- Improve readability and consistency
- Enable automatic validation
- Make communication semantics explicit
- Support security analysis and threat modeling
- Reduce ambiguity in architecture diagrams

Two-Layer Model

This convention distinguishes two separate layers:

Layer	Content	Effect
Physical Flows	<code>pull, push, write, stream</code>	Define direction and edges in the graph
Logical Annotations	<code>[req_resp], [event_ack]</code>	Mark relationships between flows — do NOT add or remove graph edges

Invariant: Flow direction defines the graph structure. Logical annotations must not introduce or remove graph edges.

Naming Format

Each dataflow label follows this structure:

```
<verb> <object> [<flow-type>]
```

Where `[flow-type]` is either:

- a **physical flow-type** (`[req]`, `[resp]`, `[cmd]`, `[event]`, `[stream]`), or
- a **logical relationship annotation** (`[req_resp]`, `[event_ack]`)

Examples:

```
pull vehicle state [req]
pull vehicle state [resp]
```

```
pull device status [req_resp]

push unlock doors [cmd]
push firmware update applied [event_ack]

write audit log

stream telemetry [stream]
```

Allowed Verbs

Only the following verbs are permitted.

Verb	Meaning	Physical Flow-Types	Logical Annotations
pull	Request/response communication	[req], [resp]	[req_resp]
push	One-way asynchronous action, command, or event	[cmd], [event]	[event_ack]
write	Persist data into a datastore	none	none
stream	Continuous or repeated data flow	[stream] (optional)	none

Deprecated / Forbidden Verbs

The following verbs must not be used:

```
send
recv
```

Also avoid synonyms such as:

```
read
fetch
query
emit
notify
publish
receive
```

The goal is to keep the vocabulary intentionally small and semantically consistent.

Rules by Verb

1. pull

Meaning

pull represents a request/response interaction.

Typical examples:

- API calls
- Database queries
- RPC communication
- Request/response protocols

Physical Flow-Types

Flow-Type	Meaning
[req]	The outgoing request half of a request/response pair
[resp]	The incoming response half of a request/response pair

Logical Annotation

Annotation	Meaning
[req_resp]	Compact notation representing a logically coupled request/response pair on a single flow

[req_resp] does not create a second graph edge. It is a modeling shorthand indicating that the flow logically represents a paired interaction.

Required Usage Patterns

A **pull** flow MUST use one of the following patterns:

Option 1 — Explicit pairing (recommended for security analysis)

```
pull <object> [req]    (A → B)
pull <object> [resp]   (B → A)
```

Both directions must be modeled as separate graph edges. The response must use the same object and reverse the direction of the request.

Option 2 — Compact representation

```
pull <object> [req_resp]
```

Represents the logically coupled pair on a single edge. Choose this when the response edge is implicit or does not require separate analysis.

Examples

Valid

```
pull vehicle state [req]
pull vehicle state [resp]

pull safety params [req]
pull safety params [resp]

pull device status [req_resp]
```

Invalid

pull vehicle state	← missing flow-type or annotation
pull vehicle state [cmd]	← [cmd] is not valid for pull
pull vehicle state [event]	← [event] is not valid for pull

2. push

Meaning

push represents a one-way asynchronous flow.

Typical examples:

- Commands
- Events
- Fire-and-forget messages
- Notifications

push is always unidirectional. Acknowledgement relationships are modeled via the **[event_ack]** logical annotation, not via a second **push** flow.

Physical Flow-Types

Flow-Type	Meaning
[cmd]	An action or instruction intended to trigger behavior
[event]	A notification that something already happened

Logical Annotation

Annotation	Meaning
[event_ack]	An event that is expected to be acknowledged via a correlated flow

[event_ack] does not represent a different type of event. It is a modeling annotation indicating that an acknowledgement relationship exists. It does NOT imply bidirectional communication and does NOT add a graph edge.

Examples

Valid Commands

```
push unlock doors [cmd]
push refresh cache [cmd]
push rotate keys [cmd]
```

Valid Events

```
push obstacle detected [event]
push vehicle moved [event]
push user authenticated [event]
```

Valid Events with acknowledgement annotation

```
push firmware update applied [event_ack]
push configuration changed [event_ack]
push safety mode activated [event_ack]
```

Invalid

```
push unlock doors           ← missing flow-type
push unlock doors [req]     ← [req] is not valid for push
push unlock doors [resp]    ← [resp] is not valid for push
```

3. write

Meaning

`write` represents persistence into a datastore or repository.

Typical examples:

- Logging
- Audit persistence
- Database writes
- State persistence

`write` is not part of the communication model. It represents a persistence operation, not a message exchange. `write` does not participate in `req_resp` or `event_ack` semantics.

Rules

`write` MUST NOT use any flow-type or logical annotation.

Examples

Valid

```
write audit log
write vehicle state
write telemetry record
```

Invalid

```
write audit log [cmd]      ← no flow-type allowed
write telemetry [req]      ← no flow-type allowed
```

4. stream

Meaning

`stream` represents a continuous or repeated producer-driven data flow.

Typical examples:

- Video feeds
- Sensor streams
- Telemetry streams

- Continuous monitoring data

`stream` flows are independent. They do not participate in `req_resp` or `event_ack` semantics. If a control channel exists alongside a stream (e.g. start/stop commands), it MUST be modeled as a separate `push [cmd]` flow. Responses to control commands (e.g. start failed, stream ended) MUST be modeled as `push [event]` or `pull [resp]` — never as part of the stream flow itself. Stream control flows belong to the `push/pull` model, not to the stream flow.

Flow-Type

`[stream]` is optional and may be added for clarity.

`stream` MUST NOT use `[req]`, `[resp]`, `[cmd]`, `[event]`, `[req_resp]`, or `[event_ack]`.

Examples

Valid

```
stream camera frames
stream telemetry
stream radar objects [stream]
```

Invalid

```
stream telemetry [req]      ← not valid for stream
stream telemetry [cmd]      ← not valid for stream
stream telemetry [event_ack] ← stream does not participate in ack
                             semantics
```

Logical Relationship Annotations

Logical annotations are syntactic macros over physical flows. They are expanded deterministically before analysis. They may appear on a diagram edge but do not change the underlying graph semantics.

Tooling requirement: Expansion MUST occur before any graph-based analysis or security evaluation (e.g. threat generation, attack path tracing, permission boundary checks).

`req_resp`

Represents a logically coupled request/response pair as a single diagram notation.

In the diagram: one edge. In analysis: expands to two directed physical edges:

```
pull <object> [req]      (A → B)
pull <object> [resp]     (B → A)
```

Both edges are analyzed independently for threats.

Valid only on **pull** flows.

```
pull <object> [req_resp]
```

event_ack

Represents a logical dependency indicating that an acknowledgement flow exists or is expected for this event.

It does not create a physical flow and does not change the graph structure. The acknowledgement is a semantic relationship only. If the acknowledgement requires explicit security analysis, model it as a separate **push** **[event]** or **pull** **[resp]** flow instead.

Valid only on **push** flows.

```
push <object> [event_ack]
```

Expansion Rules

Logical annotations are processed before any graph-based analysis or security evaluation. Processing differs by annotation type:

Annotation	Processing	Result
[req_resp]	Expands into two directed physical edges	Two graph edges for analysis
[event_ack]	Resolved as semantic dependency only	No new graph edge

req_resp expansion

```
pull <object> [req_resp]
```

expands to:


```
pull <object> [req]    (A → B)
pull <object> [resp]   (B → A)
```

Both directions are analyzed independently for threats. Each expanded edge **MUST** be analyzed independently for security and threat modeling purposes.

event_ack — no expansion

`event_ack` does NOT expand into physical flows.

It represents a semantic dependency only: the event is expected to be acknowledged, but the acknowledgement is not a physical flow in this model.

If the acknowledgement itself requires security analysis, it **MUST** be modeled as an explicit separate flow using `push [event]` or `pull [resp]` — at which point `[event_ack]` is no longer needed on the original flow.

Object Naming Guidelines

The object should describe the data or domain concept being transferred.

Prefer:

```
vehicle state
safety params
audit log
camera frames
```

Avoid:

get vehicle state	← verb embedded in object
mqtt payload	← transport detail
json message	← implementation detail
send data	← meaningless

The object should:

- be concise
- be domain-oriented
- avoid transport/protocol details
- avoid implementation-specific wording

Validator rule: The object **MUST NOT** contain transport or encoding terms. Forbidden terms include: `json`, `mqtt`, `http`, `grpc`, `rest`, `payload`, `message`, `packet`, `frame`, `buffer`.

Good Examples

```
pull user profile [req]
pull user profile [resp]

pull device status [req_resp]

push rotate keys [cmd]
push firmware update applied [event_ack]

write security audit entry

stream radar objects [stream]
```

Bad Examples

send user profile	← forbidden verb
recv user profile	← forbidden verb
pull	← missing object and flow-type
push [cmd]	← missing object
write config [cmd]	← write must not have flow-type
stream telemetry [req]	← req not valid for stream
push event [req_resp]	← req_resp not valid for push

Design Principles

This convention intentionally uses a very small vocabulary.

Benefits:

- Easier to read
- Easier to validate automatically
- Easier to model security semantics
- Consistent architecture diagrams
- Reduced ambiguity

The label communicates:

- communication style
- direction semantics
- persistence intent
- synchronization behavior

without requiring additional explanation.

Hard Invariants

1. **Flow direction defines graph structure.** Logical annotations (`[req_resp]`, `[event_ack]`) must not introduce or remove graph edges.
2. **A flow may be labeled with either a physical flow-type or a logical shorthand annotation — not both.** Logical annotations are syntactic sugar and are expanded during analysis. They may appear on the diagram edge but do not change the underlying graph semantics.
3. **`write` and `stream` do not participate in the pairing system.** They have no `req_resp` or `event_ack` semantics. `write` represents internal system state mutation and is not part of the communication graph.

Formal Grammar

```
Flow      ::= Verb Object [ Tag ]
Verb      ::= "pull" | "push" | "write" | "stream"
Object    ::= word { " " word }
Tag       ::= PhysicalTag | LogicalTag
PhysicalTag ::= "[req]" | "[resp]" | "[cmd]" | "[event]" | "[stream]"
LogicalTag  ::= "[req_resp]" | "[event_ack]"
```

Constraints not expressible in EBNF are enforced by validation rules:

- `write` MUST NOT carry any tag
- `stream` MUST NOT carry `PhysicalTag` other than `[stream]`, and MUST NOT carry `LogicalTag`
- `pull` MUST carry exactly one of: `[req]`, `[resp]`, `[req_resp]`
- `push` MUST carry exactly one of: `[cmd]`, `[event]`, `[event_ack]`

Processing Pipeline

Tools implementing this convention MUST follow this execution order:

1. Parse — tokenize and validate label syntax against grammar
2. Validate — enforce verb/tag constraint rules
3. Process — expand `[req_resp]` into physical edges only; attach `[event_ack]` as metadata only (no graph mutation)
4. Build — construct directed graph from physical flows only
5. Analyze — run threat generation, attack path tracing, boundary checks

After step 3, all logical annotations are resolved:

- `[req_resp]` → eliminated via expansion into two directed physical edges
- `[event_ack]` → removed from graph layer; carried as metadata only (no graph representation)

The graph passed to step 4 and beyond contains physical flows only.

Summary

Physical Flows

Verb	Flow-Types	Notes
<code>pull</code>	<code>[req]</code> , <code>[resp]</code>	Explicit request/response pair
<code>push</code>	<code>[cmd]</code> , <code>[event]</code>	One-way asynchronous
<code>write</code>	<code>none</code>	Persistence only
<code>stream</code>	<code>[stream]</code> (optional)	Continuous producer-driven

Logical Annotations

Annotation	Valid On	Meaning
<code>[req_resp]</code>	<code>pull</code> only	Compact request/response pair notation
<code>[event_ack]</code>	<code>push</code> only	Event with expected acknowledgement

pull

```
pull <object> [req]
pull <object> [resp]
pull <object> [req_resp]
```

push

```
push <object> [cmd]
push <object> [event]
push <object> [event_ack]
```

write

```
write <object>
```

stream

```
stream <object>  
stream <object> [stream]
```